



Caso práctico - NovaBank

Caso práctico 3 - Spring Boot



Rubén Manuel Rodríguez Chamorro

Empresa: NttData

Estudiante de Universidad de Málaga

Ingeniería de la Salud

30 de abril de 2026

Índice general

1. Introducción	3
2. Arquitectura del sistema	4
2.1. Tipo de arquitectura elegida	4
2.2. Justificación de la arquitectura	4
2.3. Responsabilidades de las capas	5
2.4. Flujo entre capas	5
2.5. Diagrama de arquitectura	6
3. Diseño de la API REST	7
3.1. Diseño general de la API	7
3.2. Recursos principales	7
3.3. Tabla de endpoints	8
3.4. DTOs utilizados en la comunicación	9
3.5. Ejemplos de peticiones	9
3.6. Validación y gestión de errores	11
3.7. Documentación de la API	11
4. Persistencia con JPA	12
4.1. Uso de JPA en el proyecto	12
4.2. Entidades persistentes	12
4.3. Relaciones entre entidades	12
4.4. Repositorios	13
4.5. Ventajas frente a JDBC	13
4.6. Diagrama del modelo de datos	14
5. Seguridad	15
5.1. Enfoque de seguridad adoptado	15
5.2. Configuración de Spring Security	15
5.2.1. Cadena de filtros de seguridad	15
5.2.2. Autenticación mediante AuthenticationProvider	16
5.3. Flujo de autenticación JWT	16

5.3.1.	Paso 1: envío de credenciales	16
5.3.2.	Paso 2: devolución del token al cliente	17
5.3.3.	Paso 3: uso del token en peticiones posteriores	17
5.4.	Validación del token en cada petición	17
6.	Testing	18
7.	Uso de Postman	19
8.	Aplicación de los principios SOLID	22
8.1.	Single Responsibility Principle	22
8.2.	Open/Closed Principle	22
8.3.	Liskov Substitution Principle	22
8.4.	Interface Segregation Principle	23
8.5.	Dependency Inversion Principle	23
9.	Refactorización respecto al Módulo 2	24

Capítulo 1

Introducción

NovaBank es un sistema de gestión bancaria sencillo que permite administrar clientes, cuentas y operaciones básicas como depósitos, retiros y transferencias.

El proyecto ha ido evolucionando por módulos. En el Módulo 1, era una aplicación de consola centrada en programación orientada a objetos y lógica de negocio.

En el Módulo 2, se reorganizó con una arquitectura por capas y persistencia en **PostgreSQL** usando **JDBC**, además de incorporar patrones de diseño y manejo de transacciones, mejorando la estructura del sistema.

En el Módulo 3, se transforma en una **API REST** con **Spring Boot**, eliminando la consola. Ahora funciona como un backend accesible por HTTP, con **arquitectura multicapa**, uso de **Spring Data JPA**, seguridad con **JWT** y **Spring Security**, y documentación mediante **Swagger/OpenAPI**.

Capítulo 2

Arquitectura del sistema

2.1. Tipo de arquitectura elegida

La arquitectura elegida para NovaBank en este módulo es una **arquitectura por capas**. Esta decisión resulta coherente con el tipo de aplicación desarrollada, ya que se trata de una API REST construida con Spring Boot que requiere separar de forma clara la exposición de endpoints, la lógica de negocio, el acceso a datos, la seguridad y la configuración general del sistema.

La adopción de una arquitectura por capas permite estructurar el proyecto de manera ordenada, favoreciendo la mantenibilidad del código y facilitando la evolución del sistema. Además, esta aproximación encaja de forma natural con el ecosistema Spring, donde la separación entre controladores, servicios y repositorios constituye una práctica ampliamente utilizada.

2.2. Justificación de la arquitectura

Para este proyecto se ha optado por una **arquitectura por capas** en lugar de una arquitectura hexagonal. La razón principal es que NovaBank, en su estado actual, es una aplicación de tamaño y complejidad moderados, donde la separación entre controladores, servicios y repositorios resulta suficiente para organizar correctamente la lógica del sistema.

La arquitectura hexagonal ofrece ventajas claras en escenarios con mayor necesidad de desacoplar completamente el dominio de la infraestructura o de soportar múltiples adaptadores de entrada y salida. Sin embargo, en este proyecto ese nivel de abstracción no era estrictamente necesario y habría añadido complejidad estructural sin aportar un beneficio proporcional al alcance real de la aplicación.

2.3. Responsabilidades de las capas

Cada paquete cumple una función concreta dentro del sistema:

- **controller**: expone los endpoints REST y gestiona las peticiones HTTP.
- **service**: implementa la lógica de negocio mediante interfaces y sus implementaciones.
- **repository**: gestiona el acceso a datos mediante Spring Data JPA.
- **model**: contiene las entidades persistentes del dominio.
- **dto**: define los objetos de entrada y salida de la API.
- **mapper**: realiza la conversión manual entre entidades y DTOs.
- **security**: centraliza la autenticación y la gestión de JWT.
- **config**: reúne la configuración general de la aplicación.
- **exception**: unifica el tratamiento de errores y excepciones.

2.4. Flujo entre capas

El funcionamiento general del sistema sigue un recorrido simple: el cliente realiza una petición HTTP a un controlador, el controlador delega la operación en un servicio, el servicio aplica la lógica de negocio y utiliza los repositorios para acceder a la base de datos. Cuando es necesario, los datos se transforman mediante DTOs y mapeadores antes de devolverse en la respuesta.

2.5. Diagrama de arquitectura

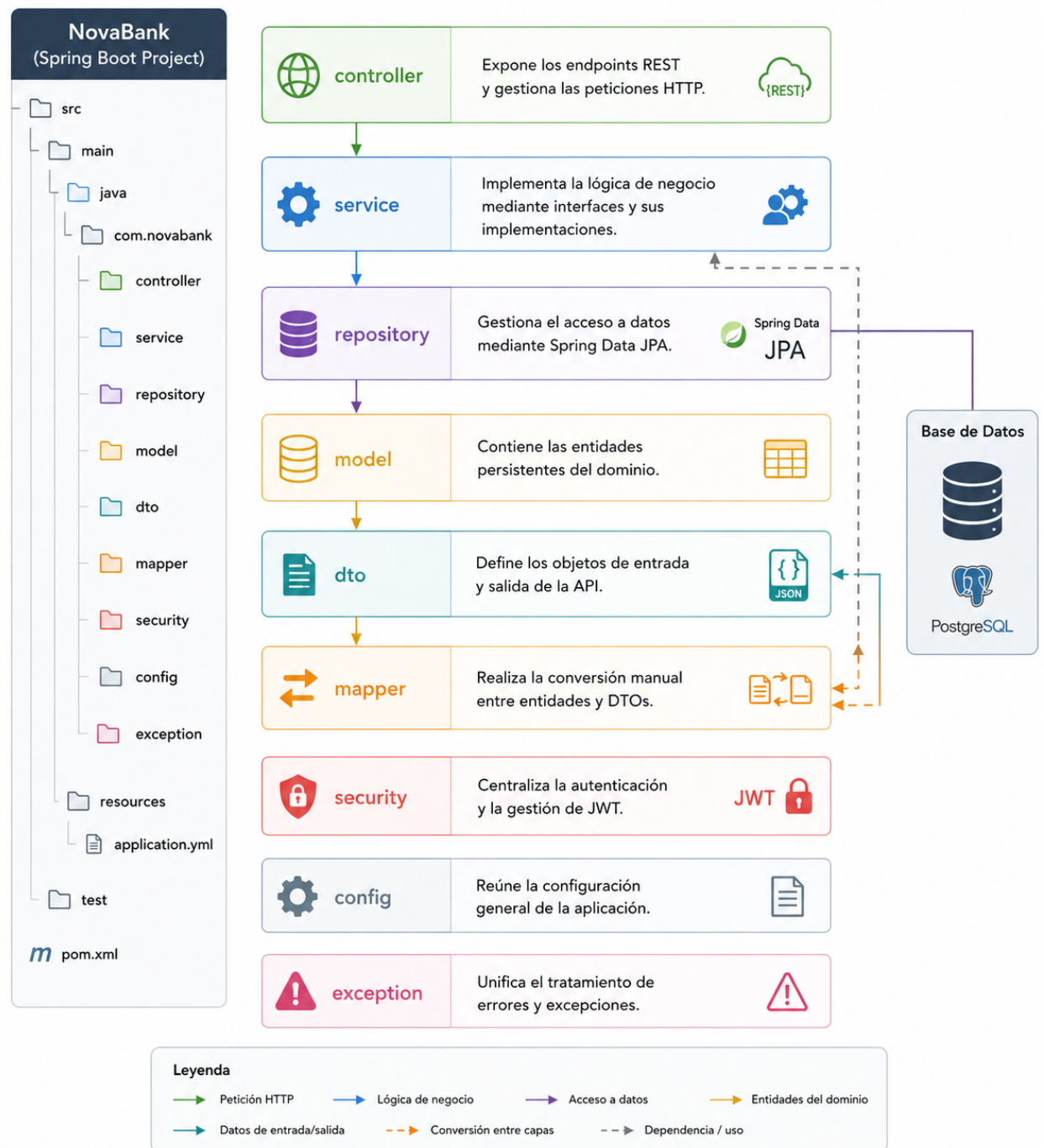


Figura 2.1: Estructura de paquetes y responsabilidades de cada capa

Capítulo 3

Diseño de la API REST

3.1. Diseño general de la API

La organización de la API responde a una estructura funcional clara, agrupando los endpoints en torno a autenticación, clientes, cuentas y operaciones.

El diseño adopta convenciones habituales en servicios REST. Las operaciones de consulta se realizan mediante métodos **GET**, mientras que la creación de recursos y la ejecución de operaciones se implementan mediante **POST**. La comunicación entre cliente y servidor se realiza en formato JSON y se apoya en DTOs para desacoplar la representación externa de la lógica interna del dominio.

Además, la API dispone de documentación interactiva mediante **Swagger/OpenAPI**, lo que facilita la exploración de endpoints, la comprobación de estructuras de entrada y salida, y la ejecución de pruebas manuales durante el desarrollo.

3.2. Recursos principales

La API se organiza en los siguientes grupos de recursos:

- **Autenticación:** acceso al sistema y obtención de token JWT.
- **Clientes:** gestión y consulta de clientes.
- **Cuentas:** creación y consulta de cuentas, saldo y movimientos.
- **Operaciones:** ejecución de depósitos, retiros y transferencias.

3.3. Tabla de endpoints

Recurso	Método HTTP	Endpoint	Descripción
Autenticación	POST	/login	Autentica al usuario y devuelve un token JWT.
Cientes	GET	/api/clientes	Devuelve el listado de clientes.
Cientes	POST	/api/clientes	Crea un nuevo cliente.
Cientes	GET	/api/clientes/{id}	Obtiene un cliente a partir de su identificador.
Cientes	GET	/api/clientes/{id}/cuentas	Devuelve las cuentas asociadas a un cliente.
Cuentas	POST	/api/cuentas	Crea una nueva cuenta bancaria.
Cuentas	GET	/api/cuentas/{id}	Obtiene una cuenta a partir de su identificador.
Cuentas	GET	/api/cuentas/{id}/saldo	Devuelve el saldo actual de una cuenta.
Cuentas	GET	/api/cuentas/{id}/movimientos	Devuelve todos los movimientos de una cuenta.
Cuentas	GET	/api/cuentas/{id}/movimientos fechaInicio=... &fechaFin=...	Devuelve los movimientos de una cuenta filtrados por rango de fechas.
Operaciones	POST	/api/operaciones/deposito	Realiza un depósito sobre una cuenta.
Operaciones	POST	/api/operaciones/retiro	Realiza un retiro sobre una cuenta.
Operaciones	POST	/api/operaciones/transferencia	Realiza una transferencia entre cuentas.

Cuadro 3.1: Endpoints principales de la API REST de NovaBank

3.4. DTOs utilizados en la comunicación

La API utiliza objetos DTO para definir de forma explícita los datos intercambiados con el cliente. Esta decisión evita exponer directamente las entidades persistentes y permite adaptar mejor las entradas y salidas a las necesidades de cada operación.

Entre los DTOs principales utilizados en la implementación se encuentran los siguientes:

- `LoginRequestDTO`: contiene las credenciales de acceso.
- `LoginResponseDTO`: devuelve el token JWT, el tipo de token y su expiración.
- `ClienteDTO`: representa los datos de un cliente.
- `CuentaDTO`: representa los datos de una cuenta bancaria.
- `MovimientoDTO`: representa un movimiento asociado a una cuenta.
- `OperacionDTO`: se utiliza en depósitos y retiros.

3.5. Ejemplos de peticiones

A continuación se muestran algunos ejemplos representativos del uso de la API.

```
1 POST http://localhost:8081/login
2 Content-Type: application/json
3
4 {
5   "username": "admin",
6   "password": "password"
7 }
```

Listing 3.1: Autenticación

```
1 GET http://localhost:8081/api/clientes
2 Authorization: Bearer <token>
```

Listing 3.2: Consultar Clientes

```
1 GET http://localhost:8081/api/cuentas/10/movimientos?fechaInicio
   =2024-01-01T00:00:00&fechaFin=2026-12-31T23:59:59
2 Authorization: Bearer <token>
```

Listing 3.3: Consulta de movimientos por rango de fechas

```

1 POST http://localhost:8081/api/operaciones/deposito
2 Authorization: Bearer <token>
3 Content-Type: application/json
4
5 {
6   "numeroCuenta": "ES9100000000000000000005",
7   "importe": 99
8 }

```

Listing 3.4: Realización de un depósito

```

1 POST http://localhost:8081/api/operaciones/transferencia
2 Authorization: Bearer <token>
3 Content-Type: application/json
4
5 {
6   "cuentaOrigen": "ES9100000000000000000001",
7   "cuentaDestino": "ES9100000000000000000005",
8   "importe": 150.00
9 }

```

Listing 3.5: Realización de una transferencia

La API devuelve respuestas estructuradas en JSON cuando corresponde, así como mensajes simples de confirmación en determinadas operaciones. En el caso del proceso de autenticación, la respuesta contiene el token JWT necesario para acceder a los endpoints protegidos.

Ejemplo de respuesta del endpoint /login:

```

1 {
2   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbGlzIm1hdCI6MTc3NzU0MjMxNCwiZXhwIjoxNzc3NjI4NzE0fQ.0kIBeeCJ2VXNox02PdQVOD2r6IAKyhh2C1G11GQE6Ec",
3   "tipo": "Bearer",
4   "expiracion": 1777628714733
5 }

```

Listing 3.6: Ejemplos de respuestas

En operaciones como depósitos, retiros o transferencias, la implementación actual devuelve un mensaje textual confirmando la ejecución de la operación. Por ejemplo:

```

1 "Deposito realizado correctamente"

```

Listing 3.7: Realización de una transferencia

Las respuestas completas de otros endpoints, como el listado de clientes, la consulta de cuentas o la obtención de movimientos, se mostrarán en las capturas incluidas en las secciones dedicadas a Swagger y Postman, donde se puede observar el comportamiento real de la API en ejecución.

3.6. Validación y gestión de errores

Los endpoints que reciben datos en el cuerpo de la petición utilizan validación mediante `@Valid`, apoyándose en restricciones declarativas dentro de los DTOs. Entre las validaciones utilizadas se encuentran `@NotBlank`, `@NotNull` y `@DecimalMin`, lo que permite rechazar peticiones incompletas o inválidas antes de ejecutar la lógica de negocio.

Además, la aplicación dispone de un `GlobalExceptionHandler`, encargado de centralizar el tratamiento de errores y devolver respuestas consistentes ante situaciones excepcionales. Esta estrategia evita dispersar la gestión de errores en los distintos controladores y mejora la coherencia de la API.

3.7. Documentación de la API

Como apoyo al diseño y validación de la API, NovaBank incorpora documentación automática mediante **Swagger/OpenAPI**. Esta documentación permite visualizar los endpoints disponibles, sus métodos HTTP, los parámetros requeridos y las estructuras de entrada y salida definidas en los DTOs.

En el documento final se incluirán capturas de Swagger que servirán como evidencia del funcionamiento y de la organización real de la API desarrollada.

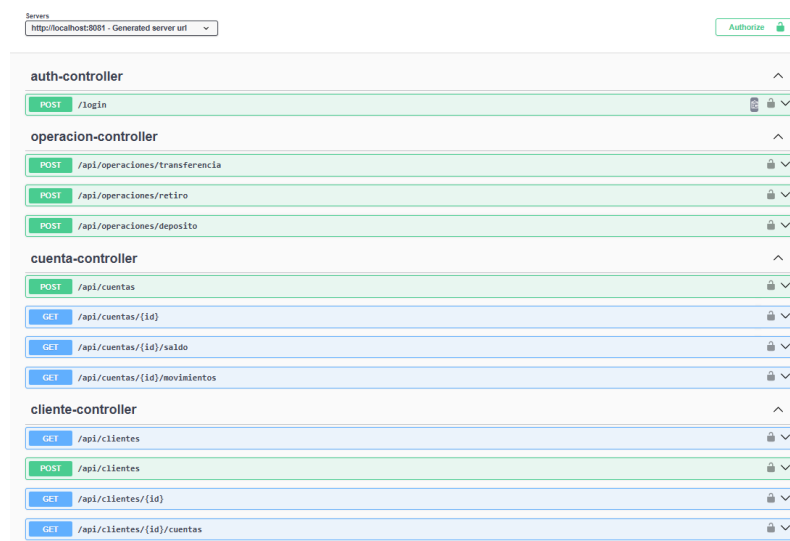


Figura 3.1: Aplicación Swagger UI

Capítulo 4

Persistencia con JPA

4.1. Uso de JPA en el proyecto

La persistencia de datos en NovaBank se ha implementado mediante **JPA** con el apoyo de **Spring Data JPA**. Esta decisión permite trabajar con un modelo orientado a objetos y delegar en el framework gran parte de las tareas relacionadas con el acceso a base de datos, como la generación de consultas básicas, la gestión del ciclo de vida de las entidades y el mapeo entre clases Java y tablas relacionales.

Frente a la aproximación utilizada en el módulo anterior con JDBC, el uso de JPA reduce el código repetitivo asociado a conexiones, sentencias SQL y mapeo manual de resultados. Además, facilita la definición de relaciones entre entidades y mejora la integración con el resto del ecosistema Spring Boot.

4.2. Entidades persistentes

El modelo persistente del sistema se compone de tres entidades principales: **Cliente**, **Cuenta** y **Movimiento**. Estas entidades representan los conceptos centrales del dominio bancario y se encuentran anotadas con `@Entity`, lo que permite a JPA gestionarlas como objetos persistentes.

4.3. Relaciones entre entidades

- Un **cliente** puede tener varias **cuentas**.
- Una **cuenta** pertenece a un único **cliente**.
- Una **cuenta** puede tener varios **movimientos**.
- Un **movimiento** pertenece a una única **cuenta**.

Desde el punto de vista de JPA, estas relaciones se implementan de la siguiente manera:

- En `Cliente` se define una relación `@OneToMany(mappedBy = 'cliente')` hacia `Cuenta`.
- En `Cuenta` se define una relación `@ManyToOne` hacia `Cliente`, utilizando la columna `cliente_id` como clave foránea.
- En `Cuenta` se define también una relación `@OneToMany(mappedBy = 'cuenta')` hacia `Movimiento`.
- En `Movimiento` se establece una relación `@ManyToOne` hacia `Cuenta`, utilizando la columna `cuenta_id`.

En las relaciones se ha utilizado `FetchType.LAZY`, lo que evita cargar automáticamente las colecciones relacionadas salvo cuando resultan necesarias. Esta decisión mejora la eficiencia y evita sobrecargar consultas simples con datos que no siempre se utilizan.

4.4. Repositorios

El acceso a la base de datos se realiza mediante interfaces que extienden `JpaRepository`. Esta aproximación permite disponer automáticamente de operaciones básicas de persistencia, como inserción, actualización, borrado y consulta por identificador, sin necesidad de implementar manualmente dichas operaciones.

4.5. Ventajas frente a JDBC

La adopción de JPA en este módulo supone una mejora clara respecto a la solución basada en JDBC utilizada anteriormente. Mientras que con JDBC era necesario gestionar manualmente conexiones, sentencias SQL y mapeo de resultados, con JPA gran parte de esa complejidad queda abstraída.

No obstante, esta abstracción no elimina la necesidad de comprender el modelo relacional subyacente, especialmente al diseñar relaciones, restricciones y consultas más específicas.

4.6. Diagrama del modelo de datos

El modelo de datos de NovaBank se articula en torno a tres entidades principales: **Cliente**, **Cuenta** y **Movimiento**. La relación entre ellas refleja la lógica del dominio bancario: un cliente puede tener varias cuentas y cada cuenta puede registrar múltiples movimientos.

La Figura ?? muestra el diagrama del modelo de datos utilizado en la aplicación, indicando las claves primarias, las claves foráneas y las relaciones principales entre entidades.

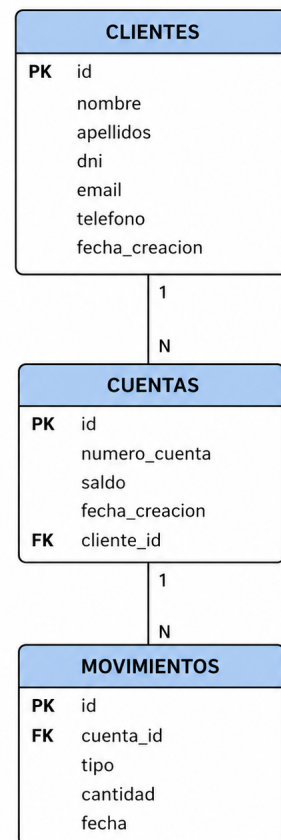


Figura 4.1: Diagrama modelo de datos

Capítulo 5

Seguridad

5.1. Enfoque de seguridad adoptado

La seguridad de NovaBank se ha implementado mediante **Spring Security** junto con autenticación basada en **JWT** (*JSON Web Token*). Este enfoque permite proteger los endpoints de la API sin mantener sesiones en servidor, lo que encaja adecuadamente con una arquitectura REST.

La decisión de utilizar JWT responde a dos motivos principales. En primer lugar, permite que la autenticación sea **stateless**, es decir, que cada petición contenga la información necesaria para ser validada sin depender de una sesión almacenada en el backend. En segundo lugar, facilita la integración con clientes como Postman, Swagger o futuras aplicaciones frontend.

5.2. Configuración de Spring Security

La configuración principal de seguridad se centraliza en la clase **SecurityConfig**. En ella se definen los componentes necesarios para autenticar usuarios, validar contraseñas, filtrar tokens JWT y proteger los recursos expuestos por la API.

5.2.1. Cadena de filtros de seguridad

La aplicación define un **SecurityFilterChain** personalizado que establece las reglas principales de seguridad:

- se desactiva la protección CSRF mediante `csrf.disable()`;
- se configura la política de sesiones como `SessionCreationPolicy.STATELESS`;
- se permite el acceso sin autenticación a `/login` y a los endpoints de Swagger;
- cualquier otro endpoint requiere autenticación previa;

- el filtro JWT se añade antes de `UsernamePasswordAuthenticationFilter`.

Esta configuración garantiza que el acceso a la API se controle mediante tokens y no mediante sesiones HTTP tradicionales.

5.2.2. Autenticación mediante `AuthenticationProvider`

La autenticación de credenciales se realiza utilizando un `DaoAuthenticationProvider`. Este proveedor se apoya en dos componentes:

- una implementación de `UserDetailsService`, encargada de cargar al usuario;
- un `PasswordEncoder`, en este caso `BCryptPasswordEncoder`, encargado de verificar la contraseña.

La clase `UserDetailsServiceImpl` define actualmente un usuario válido para la autenticación:

- usuario: `admin`
- contraseña: `password`
- rol: `ADMIN`

Si el nombre de usuario recibido no es `admin`, se lanza una excepción `UsernameNotFoundException`. Esta implementación es suficiente para el contexto académico del proyecto, aunque en un entorno real los usuarios deberían recuperarse desde base de datos.

5.3. Flujo de autenticación JWT

El proceso de autenticación de la aplicación sigue un flujo claro basado en validación de credenciales y generación posterior de un token JWT.

5.3.1. Paso 1: envío de credenciales

El usuario realiza una petición `POST` al endpoint `/login`, enviando un objeto `LoginRequestDTO` con el nombre de usuario y la contraseña.

```

1
2
3 POST http://localhost:8081/login
4 Content-Type: application/json
5
6 {
7     "username": "admin",
8     "password": "password"
9 }

```

Listing 5.1: Credenciales para el Token

5.3.2. Paso 2: devolución del token al cliente

Una vez generado, el token se devuelve al cliente encapsulado en un `LoginResponseDTO`, que contiene:

```

{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbIsIm1hdCI6MTc3NzU0MjMxNCwiZXhwIjoxNzc3NjI4NzE0fQ.0kIBeeCJ2VXNox02PdQV0D2r6IAKyhh2C1G11GQE6Ec",
  "tipo": "Bearer",
  "expiracion": 1777628714733
}

```

Figura 5.1: Devuelve el Token

5.3.3. Paso 3: uso del token en peticiones posteriores

A partir de ese momento, el cliente debe incluir el token en la cabecera `Authorization` de todas las peticiones a endpoints protegidos:

`Authorization: Bearer <token>`

5.4. Validación del token en cada petición

La validación del token JWT se realiza mediante la clase `JwtAuthenticationFilter`, que hereda de `OncePerRequestFilter`. Esto garantiza que el filtro se ejecute una sola vez por petición HTTP.

Capítulo 6

Testing

La estrategia de testing del proyecto se ha basado en combinar pruebas unitarias y pruebas de integración. De este modo, se ha validado tanto la lógica de negocio de los servicios como la interacción real con la capa de persistencia.

Las pruebas unitarias se han utilizado para comprobar de forma aislada funcionalidades como la gestión de clientes, la creación de cuentas y las operaciones bancarias principales. Por su parte, las pruebas de integración han permitido verificar el comportamiento conjunto de varias capas de la aplicación y la ejecución real de operaciones sobre base de datos, observable en los logs generados por Hibernate.

Entre las pruebas ejecutadas se encuentran `ClienteServiceImplTest`,

`CuentaServiceImplTest`, `OperacionServiceImplTest` y `OperacionIntegrationTest`.

La ejecución de `mvn test` finaliza correctamente con **27 pruebas ejecutadas, 0 fallos, 0 errores y 0 pruebas omitidas**, lo que confirma el correcto funcionamiento de las funcionalidades principales del sistema.

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.082 s --
[INFO] Running com.novabank.service.impl.ClienteServiceImplTest
2026-04-30T12:14:36.488+02:00 INFO 12372 --- [novabank-api] [      main] c.
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.148 s --
[INFO] Running com.novabank.service.impl.CuentaServiceImplTest
2026-04-30T12:14:36.617+02:00 INFO 12372 --- [novabank-api] [      main] c.
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.111 s --
[INFO] Running com.novabank.service.impl.OperacionServiceImplTest
2026-04-30T12:14:36.617+02:00 INFO 12372 --- [novabank-api] [      main] c.
2026-04-30T12:14:36.617+02:00 INFO 12372 --- [novabank-api] [      main] c.
2026-04-30T12:14:36.632+02:00 INFO 12372 --- [novabank-api] [      main] c.
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s --
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 19.254 s
[INFO] Finished at: 2026-04-30T12:14:36+02:00
[INFO] -----
```

Figura 6.1: Ejecución de `mvn test`

Capítulo 7

Uso de Postman

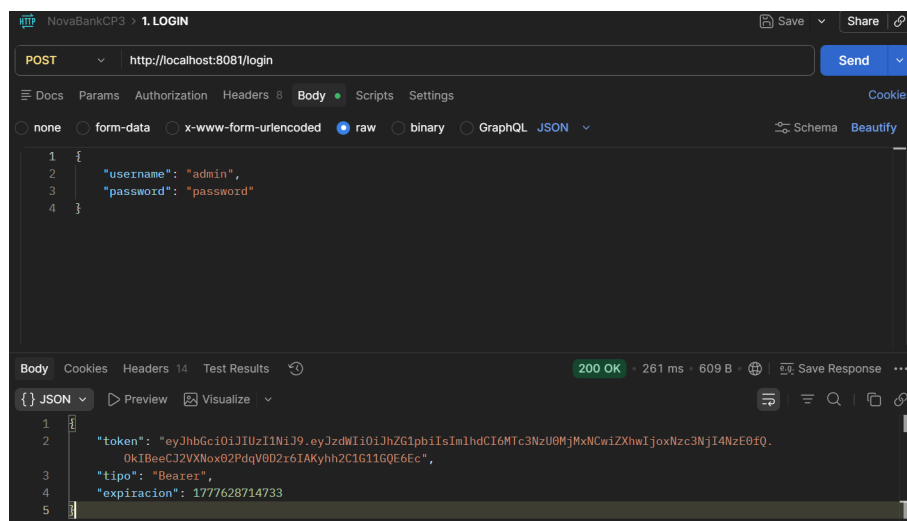


Figura 7.1: Login en Postman

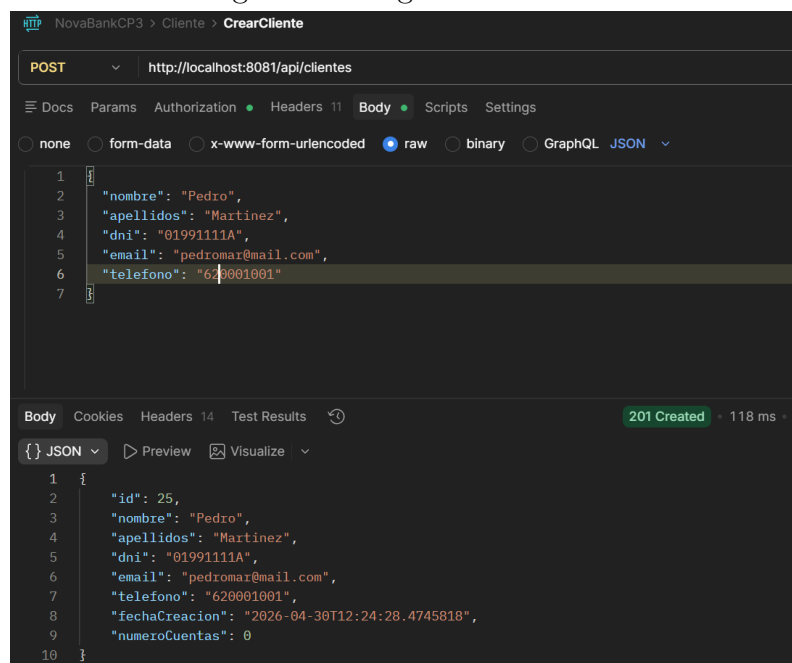


Figura 7.2: Creación de un cliente

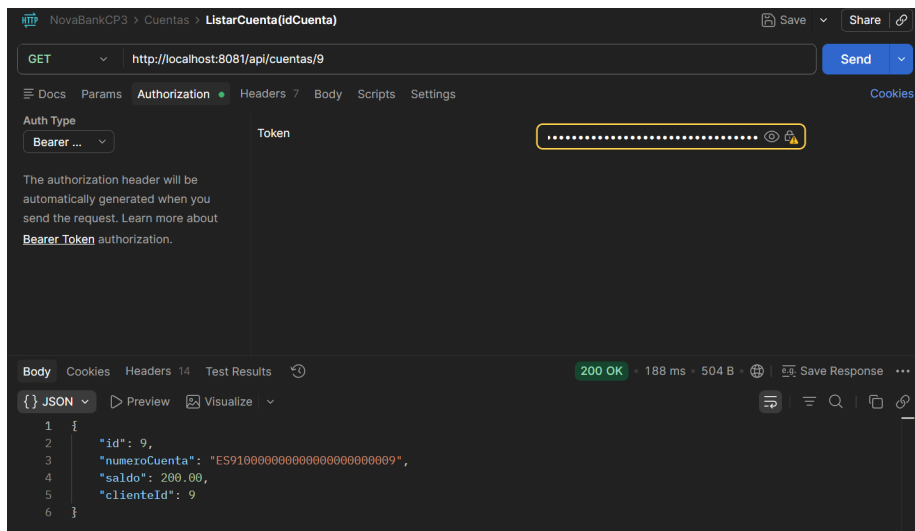


Figura 7.3: Listado del cliente 9

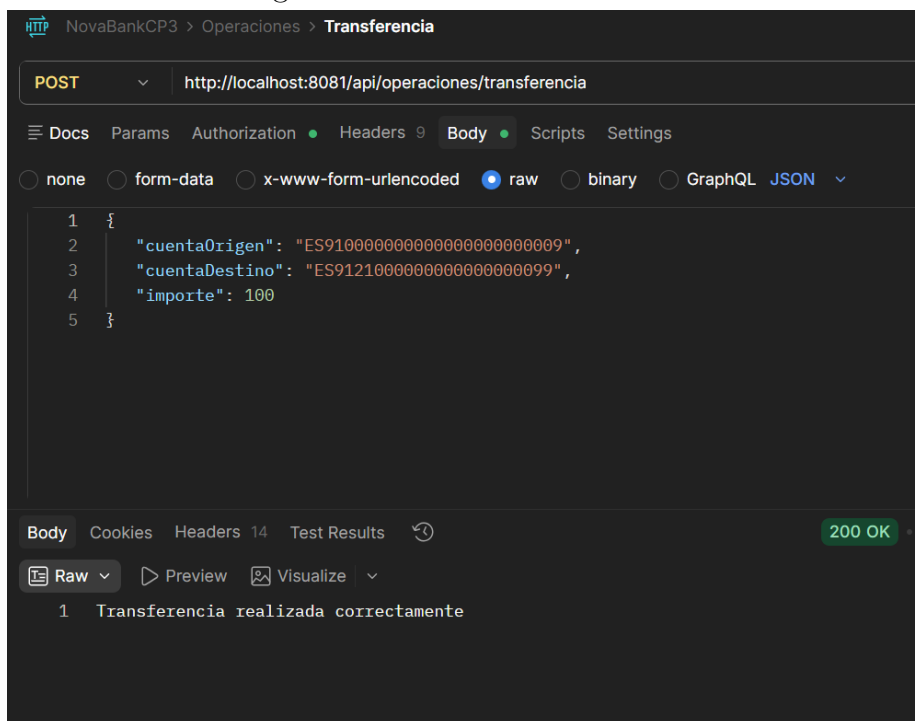


Figura 7.4: Transferencia entre dos clientes

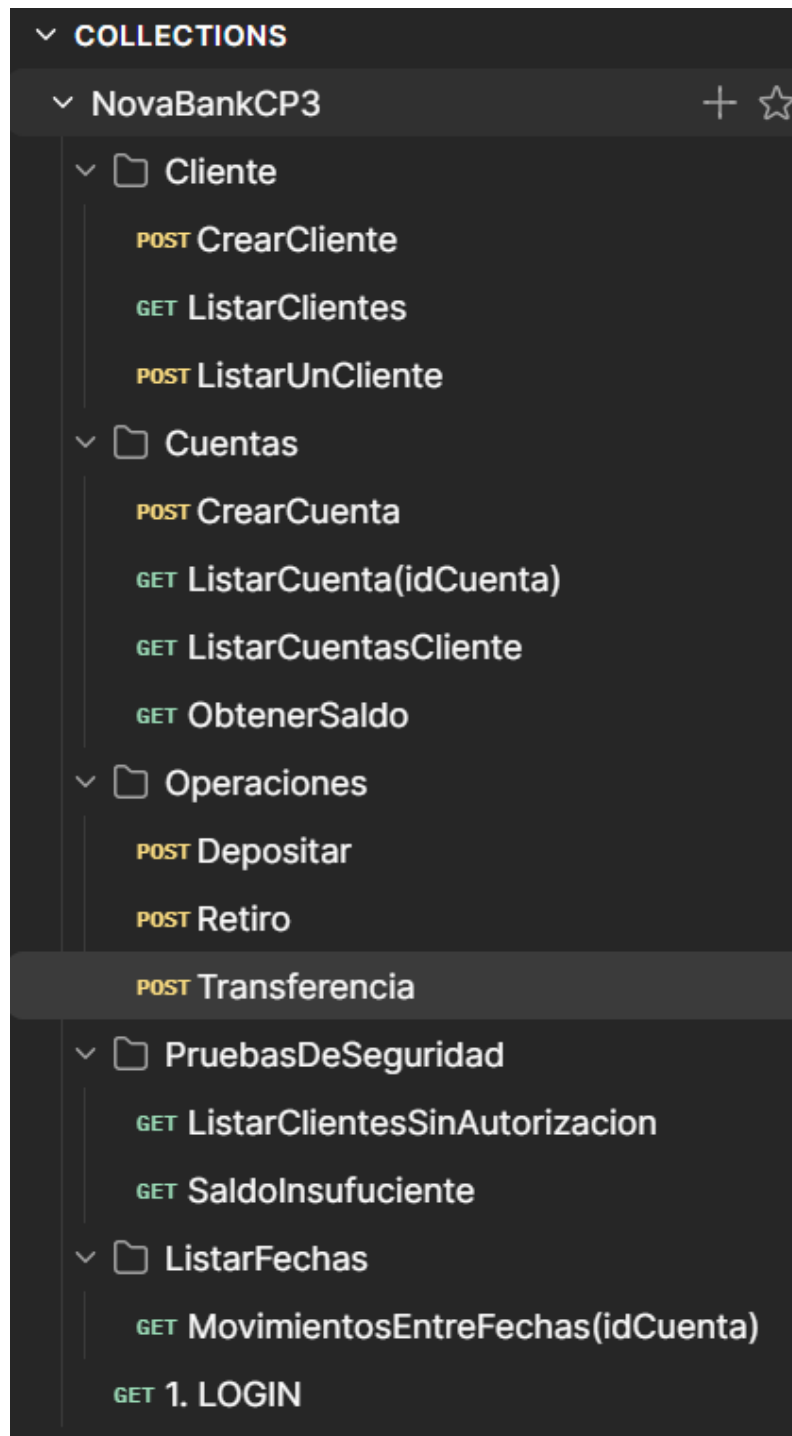


Figura 7.5: Colección completa de operaciones en Postman

Capítulo 8

Aplicación de los principios SOLID

En el proyecto NovaBank se observan varios principios SOLID aplicados en la organización del código y en la separación de responsabilidades entre controladores, servicios, repositorios y componentes de seguridad.

8.1. Single Responsibility Principle

El principio de responsabilidad única se aplica al asignar a cada clase una función concreta dentro del sistema. Por ejemplo, `ClienteController`, `CuentaController` y `OperacionController` se encargan únicamente de exponer endpoints y gestionar peticiones HTTP. La lógica de negocio se delega en clases como `ClienteServiceImpl`, `CuentaServiceImpl` y `OperacionServiceImpl`, mientras que el acceso a datos queda encapsulado en `ClienteRepository`, `CuentaRepository` y `MovimientoRepository`. Del mismo modo, la generación y validación de tokens se concentra en `JwtServiceImpl`.

8.2. Open/Closed Principle

El sistema está abierto a extensión y cerrado a modificación gracias al uso de interfaces y componentes desacoplados. Por ejemplo, `ClienteService`, `CuentaService`, `OperacionService` y `JwtService` permiten cambiar o ampliar implementaciones sin alterar los controladores que dependen de ellas. Así, `ClienteController` trabaja contra la interfaz `ClienteService`, no directamente contra una implementación concreta.

8.3. Liskov Substitution Principle

Este principio se refleja en el uso de interfaces de servicio. Las implementaciones `ClienteServiceImpl`, `CuentaServiceImpl`, `OperacionServiceImpl` y `JwtServiceImpl`

pueden sustituir a sus respectivas abstracciones sin modificar el comportamiento esperado por las clases consumidoras. Los controladores utilizan dichas abstracciones de forma transparente mediante inyección de dependencias.

8.4. Interface Segregation Principle

Las interfaces del proyecto están definidas con responsabilidades específicas y métodos ajustados a cada caso de uso. Por ejemplo, `ClienteService` solo contiene operaciones relacionadas con clientes, `CuentaService` agrupa las operaciones sobre cuentas y movimientos, y `OperacionService` se centra exclusivamente en depósitos, retiros y transferencias. Esto evita interfaces demasiado grandes o forzar a una clase a implementar métodos que no necesita.

8.5. Dependency Inversion Principle

El principio de inversión de dependencias se aplica mediante el uso de inyección de dependencias y programación contra abstracciones. Los controladores dependen de interfaces de servicio, como `ClienteService` o `OperacionService`, en lugar de depender directamente de sus implementaciones. Del mismo modo, la autenticación en `AuthController` utiliza la abstracción `JwtService`. Spring resuelve automáticamente estas dependencias, reduciendo el acoplamiento y mejorando la mantenibilidad del sistema.

Capítulo 9

Refactorización respecto al Módulo 2

Respecto al Módulo 2, NovaBank ha experimentado una refactorización orientada a modernizar tanto su arquitectura como su forma de exposición al cliente. El cambio principal ha sido la migración a **Spring Boot**, dejando atrás la aplicación de consola para construir una **API REST** estructurada y accesible mediante HTTP.

En la capa de persistencia, se ha sustituido el acceso a base de datos mediante **JDBC** por un modelo basado en **Spring Data JPA**, lo que ha permitido trabajar con entidades, repositorios y mapeo objeto-relacional en lugar de gestionar manualmente gran parte de las operaciones de acceso a datos.

Además, se ha incorporado un sistema de **seguridad con Spring Security y JWT**, con el objetivo de proteger los endpoints y controlar el acceso a los recursos de la API. Finalmente, se ha añadido **documentación automática con Swagger/OpenAPI**, facilitando la exploración y prueba de los endpoints durante el desarrollo y la validación del sistema.

Enlace al repositorio: https://github.com/Rvbenrch/SpringBoot_NovaBank/tree/develop

Bibliografía

- [1] Spring Documentation. *Spring Boot Reference Documentation*. Disponible en: <https://spring.io/projects/spring-boot>
- [2] Spring Documentation. *Spring Security Reference*. Disponible en: <https://spring.io/projects/spring-security>
- [3] Spring Documentation. *Spring Data JPA*. Disponible en: <https://spring.io/projects/spring-data-jpa>
- [4] Springdoc OpenAPI. *springdoc-openapi Documentation*. Disponible en: <https://springdoc.org/>
- [5] **Rodríguez Chamorro, Rubén Manuel**. *SpringBoot_NovaBank*. Repositorio en GitHub. Disponible en: https://github.com/Rvbenrch/SpringBoot_NovaBank/tree/develop